

Java + Spring Boot

Interview Questions & Answers

Top 55 Most-Asked Questions | 3 Years Experience

Core Java • Advanced Java • Collections • Multithreading

Spring Boot • JPA/Hibernate • REST API • Microservices • Redis

Includes code examples & interview tips for every question

Section breakdown: Java & Adv Java (17Q) • Spring Boot (9Q) • JPA (5Q) • REST (4Q) • Microservices (5Q) • Redis (3Q) • System Design (4Q)

SECTION 1 — Core Java

Q1–Q10 | Most asked — fundamentals interviewers never skip

Q1.

What is the difference between HashMap, LinkedHashMap, and TreeMap?

- HashMap: no ordering guarantee, O(1) get/put, allows one null key.
- LinkedHashMap: maintains insertion order (or access order), O(1) get/put. Good for LRU cache.
- TreeMap: sorted by natural order or Comparator, O(log n) get/put. Implements NavigableMap.

```
Map<String,Integer> hash    = new HashMap<>();           // no order
Map<String,Integer> linked = new LinkedHashMap<>();      // insertion order
Map<String,Integer> tree    = new TreeMap<>();           // sorted by key

// LRU cache using LinkedHashMap
new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry e) {
        return size() > MAX_SIZE;
    }
};
```

■ *Interview tip: TreeMap is backed by a Red-Black tree. Say this — it signals depth of knowledge.*

Q2.

What is the difference between HashMap and ConcurrentHashMap?

HashMap is not thread-safe — concurrent modifications can cause infinite loops or data loss. ConcurrentHashMap (Java 8+) uses CAS + synchronized on individual bins, allowing full concurrent reads and high-throughput writes without locking the whole map.

```
// HashMap – unsafe in multi-threaded code
Map<String,Integer> map = new HashMap<>();

// ConcurrentHashMap – thread-safe, no null keys/values
Map<String,Integer> safe = new ConcurrentHashMap<>();
safe.computeIfAbsent("orders", k -> loadFromDB(k));
safe.merge("count", 1, Integer::sum); // atomic increment
```

■ *Interview tip: ConcurrentHashMap does NOT allow null keys or values — common trick question.*

Q3.**How does ArrayList differ from LinkedList — when to use which?**

- ArrayList: backed by dynamic array. $O(1)$ random access, $O(n)$ insert/delete in middle. Better for read-heavy operations.
- LinkedList: doubly-linked list. $O(1)$ insert/delete at head/tail, $O(n)$ random access. Better for frequent insertions/deletions.
- In practice: ArrayList is almost always faster due to cache locality — prefer it unless you have a specific need for $O(1)$ deque operations.

```
List<String> list = new ArrayList<>(); // random access fast
Deque<String> deque = new LinkedList<>(); // queue/stack ops

// ArrayDeque is faster than LinkedList for queue use case
Deque<String> fast = new ArrayDeque<>();
```

■ *Interview tip: Mention that ArrayDeque beats LinkedList for queue operations — shows real-world awareness.*

Q4.**Explain the Java Collections hierarchy — List, Set, Map, Queue.**

- Iterable → Collection → List (ArrayList, LinkedList, Vector)
- Iterable → Collection → Set (HashSet, LinkedHashSet, TreeSet)
- Iterable → Collection → Queue (ArrayDeque, PriorityQueue, LinkedList)
- Map (HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap) — NOT part of Collection

```
// Key implementations to know
List<?> list = new ArrayList<>(); // ordered, duplicates OK
Set<?> set = new HashSet<>(); // unordered, no duplicates
Set<?> sorted = new TreeSet<>(); // sorted, no duplicates
Queue<?> pq = new PriorityQueue<>(); // min-heap by default
Map<?, ?> map = new HashMap<>(); // key-value, unique keys
```

■ *Interview tip: Interviewers love asking: 'Which Collection would you use for...' — practice matching use-cases.*

Q5.**What is the difference between String, StringBuilder, and StringBuffer?**

- String: immutable. Every modification creates a new object. Stored in String Pool.
- StringBuilder: mutable, NOT thread-safe. Preferred for single-threaded string manipulation.
- StringBuffer: mutable, thread-safe (synchronized). Use only when shared across threads.

```
// String – immutable (bad in loops)
String s = "";
for (int i=0; i<1000; i++) s += i; // creates 1000 objects!

// StringBuilder – preferred
StringBuilder sb = new StringBuilder();
for (int i=0; i<1000; i++) sb.append(i); // single object
String result = sb.toString();
```

■ *Interview tip: Java compiler auto-converts + concatenation in simple expressions to StringBuilder — but not in loops.*

Q6.**What is the difference between == and .equals(), and the hashCode contract?**

== compares references (memory addresses). .equals() compares logical equality. The hashCode contract: if two objects are equal via .equals(), they MUST have the same hashCode. The reverse is not required (hash collisions are allowed).

```
String a = new String("hello");
String b = new String("hello");
a == b;           // false - different objects
a.equals(b);      // true  - same content

// Always override BOTH together
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User)) return false;
    return Objects.equals(id, ((User)o).id);
}
@Override public int hashCode() {
    return Objects.hash(id);
}
```

■ *Interview tip: If you override only equals without hashCode, your object will break in HashMap/HashSet.*

Q7.**What are Java 8 Streams — intermediate vs terminal operations?**

Streams are lazy pipelines. Intermediate ops (filter, map, sorted, distinct, flatMap) return a new Stream and are NOT executed until a terminal op is called. Terminal ops (collect, forEach, count, reduce, findFirst, anyMatch) trigger the pipeline.

```
List<String> names = List.of("Alice", "Bob", "Anna", "Charlie");

// Intermediate + terminal
Map<Integer, List<String>> byLength = names.stream()
    .filter(n -> n.length() > 3)           // intermediate
    .sorted()                             // intermediate
    .collect(Collectors.groupingBy(String::length)); // terminal

// reduce
int sum = IntStream.rangeClosed(1,10).reduce(0, Integer::sum);

// flatMap
List<Integer> flat = List.of(List.of(1,2), List.of(3,4))
    .stream().flatMap(Collection::stream).collect(toList());
```

■ *Interview tip: Practice groupingBy, partitioningBy, and joining collectors — they appear in almost every Java interview.*

Q8.**What is the difference between Comparable and Comparator?**

- Comparable (java.lang): defines natural ordering inside the class via compareTo(). E.g. String, Integer already implement it.
- Comparator (java.util): defines external/custom ordering without modifying the class. Can have multiple Comparators for the same class.

```
// Comparable – natural order
class Employee implements Comparable<Employee> {
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name);
    }
}

// Comparator – custom order (lambda)
List<Employee> emps = getEmployees();
emps.sort(Comparator.comparing(Employee::getSalary)
    .thenComparing(Employee::getName));
```

■ *Interview tip: Comparator.comparing() with method references is the modern clean way — always write it this way.*

Q9.**Explain try-with-resources and the AutoCloseable interface.**

Try-with-resources (Java 7+) automatically closes resources that implement AutoCloseable or Closeable after the try block, even if an exception occurs. Eliminates the need for finally blocks for cleanup.

```
// Old style – verbose and error-prone
Connection conn = null;
try { conn = getConn(); use(conn); }
finally { if (conn!=null) conn.close(); }

// Try-with-resources – clean and safe
try (Connection conn = getConn();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.executeQuery();
} // conn and ps auto-closed here

// Custom AutoCloseable
class Timer implements AutoCloseable {
    long start = System.currentTimeMillis();
    public void close() {
        System.out.println(System.currentTimeMillis()-start+"ms");
    }
}
```

■ *Interview tip: Multiple resources are closed in reverse declaration order — a detail that shows depth.*

Q10.**What is the difference between checked and unchecked exceptions?**

- Checked (extends `Exception`): compiler forces you to handle or declare them. E.g. `IOException`, `SQLException`, `ClassNotFoundException`.
- Unchecked (extends `RuntimeException`): not checked at compile time. E.g. `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException`.
- Spring best practice: use unchecked exceptions for business logic errors to keep code clean.

```
// Checked – must declare or catch
public void readFile(String path) throws IOException {
    Files.readAllBytes(Paths.get(path));
}

// Custom unchecked – Spring style
public class OrderNotFoundException extends RuntimeException {
    public OrderNotFoundException(Long id) {
        super("Order not found: " + id);
    }
}
```

■ *Interview tip: Spring's `DataAccessException` hierarchy is all unchecked — mention this as real-world example.*

SECTION 2 — Advanced Java

Q11–Q20 | Multithreading, JVM, Generics, Design Patterns

Q11.**What is the difference between synchronized, volatile, and AtomicInteger?**

- `synchronized`: ensures mutual exclusion — only one thread executes the block at a time. Guarantees visibility + atomicity.
- `volatile`: guarantees visibility (changes are immediately visible to all threads) but NOT atomicity. Good for simple flags.
- `AtomicInteger`: lock-free, thread-safe integer operations using CAS (Compare-And-Swap). Better performance than `synchronized` for counters.

```
// volatile – visibility only
private volatile boolean running = true;
void stop() { running = false; } // visible immediately

// AtomicInteger – atomic increment
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // thread-safe, no lock

// synchronized – full mutual exclusion
synchronized void transfer(int amount) {
    balance -= amount; // both visibility + atomicity
}
```

■ *Interview tip: Use `AtomicInteger` over `synchronized` for counters — it's lock-free and much faster under contention.*

Q12.**What is the difference between wait(), notify(), and sleep()?**

- wait(): releases the monitor lock and waits — must be called inside synchronized block. Called on an Object.
- notify() / notifyAll(): wakes waiting thread(s) — must be inside synchronized block.
- sleep(): pauses the current thread for a duration — does NOT release the lock. Called on Thread.

```
// Producer-Consumer using wait/notify
synchronized void produce(Item item) throws InterruptedException {
    while (queue.size() == MAX) wait(); // release lock + wait
    queue.add(item);
    notifyAll(); // wake consumers
}
synchronized Item consume() throws InterruptedException {
    while (queue.isEmpty()) wait();
    Item item = queue.poll();
    notifyAll();
    return item;
}
```

■ *Interview tip: Always use wait() inside a while loop, not if — to guard against spurious wakeups.*

Q13.**What is the ExecutorService and how does thread pool work?**

ExecutorService manages a pool of threads for executing tasks asynchronously. Avoids overhead of creating/destroying threads per task.

```
// Fixed thread pool
ExecutorService pool = Executors.newFixedThreadPool(10);

// Submit Callable – returns Future
Future<Integer> future = pool.submit(() -> {
    return expensiveCalc();
});
Integer result = future.get(5, TimeUnit.SECONDS); // blocking

// CompletableFuture – non-blocking chain
CompletableFuture.supplyAsync(() -> fetchUser(id), pool)
    .thenApply(user -> enrichUser(user))
    .thenAccept(user -> sendEmail(user))
    .exceptionally(ex -> { log(ex); return null; });

pool.shutdown(); // always shutdown
```

■ *Interview tip: Always shutdown ExecutorService — resource leak is a common bug. Use try-finally or shutdown hooks.*

Q14.**What is deadlock and how do you prevent it?**

Deadlock occurs when two or more threads wait for each other to release locks, causing all to block indefinitely. Four necessary conditions: mutual exclusion, hold and wait, no preemption, circular wait.

```
// Deadlock – Thread 1 holds lockA, waits for lockB
//           Thread 2 holds lockB, waits for lockA

// Prevention: always acquire locks in same order
void transfer(Account from, Account to, int amt) {
    Account first = from.id < to.id ? from : to;
    Account second = from.id < to.id ? to : from;
    synchronized(first) {
        synchronized(second) {
            from.debit(amt);
            to.credit(amt);
        }
    }
}
```

■ *Interview tip: Mention tryLock() with timeout from ReentrantLock as another strategy — avoids infinite waiting.*

Q15.**Explain Java Generics — bounded wildcards, type erasure.**

- Generics provide compile-time type safety. Type erasure removes generic type info at runtime.
- (upper bound): read-only. Accepts T and subtypes.
- (lower bound): write-friendly. Accepts T and supertypes.
- PECS rule: Producer Extends, Consumer Super.

```
// Upper bound – read from list
double sum(List<? extends Number> list) {
    return list.stream().mapToDouble(Number::doubleValue).sum();
}
sum(List.of(1, 2.5, 3L)); // works for Integer, Double, Long

// Lower bound – write to list
void addNumbers(List<? super Integer> list) {
    list.add(1); list.add(2);
}

// Type erasure – at runtime List<String> == List<Integer>
List<String> s = new ArrayList<>();
List<Integer> i = new ArrayList<>();
s.getClass() == i.getClass(); // true at runtime!
```

■ *Interview tip: PECS — Producer Extends, Consumer Super. Memorise it — it comes up in almost every senior Java round.*

Q16.**What are Java 8 Optional and how should you use it?**

Optional is a container that may or may not hold a value. It forces explicit handling of null cases and eliminates NullPointerException when used correctly.

```
// Creating
Optional<User> user = Optional.of(getUser(id)); // NPE if null
Optional<User> safe = Optional.ofNullable(getUser(id)); // safe
Optional<User> empty = Optional.empty();

// Consuming – prefer functional style
user.map(User::getEmail)
    .filter(e -> e.contains("@"))
    .ifPresent(e -> sendEmail(e));

// Get with default
String email = user.map(User::getEmail)
    .orElse("no-reply@default.com");

// Throw if absent
User u = user.orElseThrow(() ->
    new UserNotFoundException(id));
```

■ *Interview tip: Never use `Optional.get()` without `isPresent()` — and never use `Optional` as a method parameter or field.*

Q17.**What are common Design Patterns used in Java / Spring Boot?**

- Singleton: one instance per JVM — Spring beans are singletons by default.
- Factory: Spring's BeanFactory creates beans based on configuration.
- Builder: used in StringBuilder, Lombok @Builder, JPA query builders.
- Proxy: Spring AOP uses proxies for @Transactional, @Cacheable.
- Observer: ApplicationEventPublisher / @EventListener in Spring.
- Strategy: inject different implementations of an interface at runtime.

```
// Strategy pattern – inject by Spring
interface PaymentStrategy { void pay(BigDecimal amount); }

@Service("stripe") class StripePayment implements PaymentStrategy {}
@Service("paypal") class PaypalPayment implements PaymentStrategy {}

@Service class CheckoutService {
    @Autowired
    Map<String, PaymentStrategy> strategies; // injected by Spring

    void checkout(String method, BigDecimal amt) {
        strategies.get(method).pay(amt);
    }
}
```

■ *Interview tip: Injecting a Map to select strategy by name is a powerful Spring pattern — impress interviewers with this.*

SECTION 3 — Spring Boot

Q21–Q29 | Core container, AOP, Security, Configuration

Q21.**How does Spring Boot auto-configuration work?**

@SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan. Auto-config scans META-INF/spring/autoconfigure.imports for @Configuration classes with @Conditional* annotations that apply only when conditions match (class on classpath, bean missing, property set).

```
@Configuration
@ConditionalOnClass(DataSource.class) // only if JDBC on classpath
@ConditionalOnMissingBean(DataSource.class) // only if not already defined
public class DataSourceAutoConfiguration { ... }

// Disable specific auto-config
@SpringBootApplication(
    exclude = {DataSourceAutoConfiguration.class})
public class App {}
```

■ *Interview tip: Run with --debug to see which auto-configs matched/did not match — very useful for debugging.*

Q22.**What is Spring AOP and how does @Transactional use it?**

Spring AOP wraps beans in JDK dynamic proxies (interface-based) or CGLIB proxies (class-based). @Transactional is implemented as an AOP advice that intercepts method calls to begin/commit/rollback transactions.

```
// @Transactional only works via proxy
// Self-invocation BYPASSES the proxy!
@Service
public class OrderService {
    @Transactional
    public void outer() {
        inner(); // WRONG – bypasses transaction!
    }
    @Transactional(propagation=REQUIRES_NEW)
    public void inner() { ... }
}
// Fix: inject self, or use separate service class
```

■ *Interview tip: @Transactional only works on public methods called via Spring proxy — self-invocation is the #1 gotcha.*

Q23.**What is the difference between @Component, @Service, @Repository, @Controller?**

- All are @Component specializations triggering component scan.
- @Repository: adds PersistenceExceptionTranslation — converts DB exceptions to Spring's DataAccessException hierarchy.
- @Service: marks business layer, no extra Spring behavior — purely semantic.
- @RestController = @Controller + @ResponseBody — returns JSON/XML directly.

```
@Repository // persistence layer + exception translation
public class UserRepository { ... }

@Service // business logic
public class UserService { ... }

@RestController // HTTP handler + JSON response
@RequestMapping("/api/users")
public class UserController { ... }
```

■ *Interview tip: The exception translation in @Repository is a commonly missed technical detail — say it explicitly.*

Q24.**How do you handle exceptions globally in Spring Boot?**

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(404);
        pd.setDetail(ex.getMessage());
        return pd; // RFC 7807 format (Spring Boot 3+)
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String,String> handleValidation(
        MethodArgumentNotValidException ex) {
        return ex.getBindingResult().getFieldErrors().stream()
            .collect(toMap(FieldError::getField,
                FieldError::getDefaultMessage));
    }
}
```

■ *Interview tip: ProblemDetail (RFC 7807) is Spring Boot 3.x standard — mention it to show you're current.*

Q25.**What is Spring Bean scope and what are the common gotchas?**

- singleton (default): one instance per ApplicationContext — shared, stateless beans.
- prototype: new instance every time — stateful beans.
- request / session: one per HTTP request/session — web apps only.
- Gotcha: injecting a prototype bean into a singleton — the prototype is only created once!

```
// Problem: prototype injected into singleton
@Service @Scope("prototype")
class ReportBuilder { ... }

@Service // singleton — gets same ReportBuilder every time!
class ReportService {
    @Autowired ReportBuilder builder; // WRONG
}

// Fix: use ApplicationContext or @Lookup
@Lookup
protected ReportBuilder getBuilder() { return null; }
```

■ *Interview tip: The singleton + prototype injection bug is a favourite interview question — demonstrate awareness.*

Q26.**How do you secure a Spring Boot REST API with JWT?**

```
// Filter extracts & validates JWT from Authorization header
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String token = req.getHeader("Authorization");
        if (token != null && token.startsWith("Bearer ")) {
            String jwt = token.substring(7);
            if (jwtUtil.isValid(jwt)) {
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        jwtUtil.getUser(jwt), null,
                        jwtUtil.getRoles(jwt));
                SecurityContextHolder.getContext()
                    .setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

■ *Interview tip: Store JWT secret in environment variables / Vault — never hardcode. Interviewers watch for this.*

Q27.**What is Spring Actuator and what endpoints does it expose?**

- /actuator/health — application and dependency health status.
- /actuator/metrics — JVM, HTTP, datasource, custom metrics.
- /actuator/info — build info, git commit, custom properties.
- /actuator/env — environment properties (sensitive — restrict in prod!).
- /actuator/loggers — change log levels at runtime without restart.

```
# application.yml
management:
  endpoints:
    web.exposure.include: health,info,metrics
  endpoint:
    health.show-details: when_authorized

# Custom health indicator
@Component
class ExternalApiHealth implements HealthIndicator {
    public Health health() {
        return externalApi.isUp()
            ? Health.up().build()
            : Health.down().withDetail("reason","timeout").build();
    }
}
```

■ *Interview tip: Never expose /actuator/env or /actuator/shutdown in production — critical security risk.*

SECTION 4 — JPA / Hibernate

Q30–Q34 | N+1, Locking, Transactions

Q30.**What is the N+1 query problem and how do you fix it?**

N+1 occurs when loading N parent entities triggers N extra queries for child entities. 100 orders → 1 query + 100 item queries = 101 DB round trips.

```
// BAD — N+1
orderRepo.findAll()           // 1 query
    .forEach(o->o.getItems()); // N queries

// FIX 1 — JOIN FETCH
@Query("SELECT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();

// FIX 2 — EntityGraph
@EntityGraph(attributePaths={"items","customer"})
List<Order> findByStatus(OrderStatus status);

// FIX 3 — @BatchSize for lazy collections
@BatchSize(size=25)
@OneToMany private List<Item> items;
```

■ *Interview tip: Enable show-sql + format_sql in dev to catch N+1 early. Hibernate Statistics gives exact query counts.*

Q31.**What is the difference between FetchType.LAZY and EAGER? Best practice?**

- EAGER: loads associated entities immediately in the same query. Default for @ManyToOne, @OneToOne.
- LAZY: loads only when accessed. Default for @OneToMany, @ManyToMany.
- Best practice: always declare LAZY on all associations. Use JOIN FETCH / @EntityGraph when you need the data.

```
@Entity public class Order {  
    // Override EAGER default to LAZY  
    @ManyToOne(fetch = FetchType.LAZY)  
    private Customer customer;  
  
    @OneToMany(mappedBy="order", fetch=FetchType.LAZY)  
    private List<Item> items; // default LAZY  
}
```

■ *Interview tip: LazyInitializationException means you accessed a LAZY relation outside a transaction — open-session-in-view is the bad fix; load eagerly in service layer is the right fix.*

Q32.**What is Optimistic vs Pessimistic locking in JPA?**

- Optimistic: @Version field — no DB lock; checks version at commit. Throws OptimisticLockException on conflict. Best for low-contention reads.
- Pessimistic: DB-level SELECT FOR UPDATE lock. Blocks concurrent writes. Best for high-contention financial transactions.

```
// Optimistic – add @Version  
@Entity public class Inventory {  
    @Version private Long version; // JPA manages this  
    private int stock;  
}  
  
// Pessimistic – lock at query time  
@Lock(LockModeType.PESSIMISTIC_WRITE)  
@Query("SELECT i FROM Inventory i WHERE i.productId = :id")  
Optional<Inventory> lockByProduct(@Param("id") Long id);
```

■ *Interview tip: For e-commerce stock deduction, use pessimistic write lock — prevents overselling under high load.*

Q33.**What are @Transactional propagation levels and when do you use them?**

- REQUIRED (default): join existing transaction or create new. Safe for most cases.
- REQUIRES_NEW: always create new transaction, suspend existing. Use for audit logs that must commit even if outer transaction rolls back.
- NESTED: creates a savepoint within existing transaction. Partial rollback possible.
- NEVER: throws exception if a transaction already exists. Use for read operations that must not be transactional.

```
@Transactional(propagation = Propagation.REQUIRED) // default
public void placeOrder(Order o) {
    saveOrder(o);
    auditService.log(o); // runs in SAME transaction
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void log(Order o) {
    // NEW transaction – commits even if outer rolls back
    auditRepo.save(new AuditLog(o));
}
```

■ Interview tip: *REQUIRES_NEW* for audit logging is the most common real-world use case — always mention it.

Q34.**What is the difference between JPQL, Criteria API, and Native Query?**

- JPQL: object-oriented query language, entity/field names not table/column names. Portable across DBs.
- Criteria API: type-safe, programmatic query building. Verbose but refactor-safe.
- Native Query: raw SQL — use for DB-specific features, complex joins, or performance-critical queries.

```
// JPQL
@Query("SELECT o FROM Order o WHERE o.status=:s AND o.total > :min")
List<Order> findByStatusAndMinTotal(@Param("s") Status s,
                                   @Param("min") BigDecimal min);

// Native Query
@Query(value="SELECT * FROM orders WHERE YEAR(created_at)=:yr",
      nativeQuery=true)
List<Order> findByYear(@Param("yr") int year);

// Projection (efficient – fetch only needed columns)
interface OrderSummary { Long getId(); BigDecimal getTotal(); }
List<OrderSummary> findByCustomerId(Long customerId);
```

■ Interview tip: *Projections (interfaces with getters)* are under-used but highly efficient — only selected columns are fetched.

SECTION 5 — REST API Design

Q35–Q38 | Best practices, Validation, Versioning

Q35.**What are REST API best practices for production-grade APIs?**

- Use nouns for resources: /orders, not /getOrders.
- HTTP methods: GET (read), POST (create), PUT (full update), PATCH (partial), DELETE.
- Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 500.
- Versioning: /api/v1/orders or via Accept header.
- Pagination: ?page=0&size=20&sort=createdAt,desc.
- Consistent error response with timestamp, status, message, path.
- Idempotency keys for POST endpoints that create financial transactions.

■ *Interview tip: Mention HATEOAS and RFC 7807 ProblemDetail for extra credit — these show API maturity.*

Q36.**How do you implement request validation in Spring Boot?**

```
// DTO with Bean Validation
public class CreateOrderRequest {
    @NotBlank(message="Customer ID required")
    private String customerId;

    @DecimalMin(value="0.01", message="Amount must be positive")
    private BigDecimal amount;

    @NotEmpty @Valid
    private List<@NotNull OrderItem> items;

    @Future(message="Must be a future date")
    private LocalDate deliveryDate;
}

// Controller
@PostMapping("/orders")
public ResponseEntity<OrderDto> create(
    @Valid @RequestBody CreateOrderRequest req) {
    return ResponseEntity.status(201)
        .body(orderService.create(req));
}
```

■ *Interview tip: @Validated (Spring) vs @Valid (JSR-380): use @Validated on class level for group validation.*

Q37.**How do you implement pagination and sorting in Spring Boot?**

```
@GetMapping("/orders")
public Page<OrderDto> getOrders(
    @RequestParam(defaultValue="0") int page,
    @RequestParam(defaultValue="20") int size,
    @RequestParam(defaultValue="createdAt") String sort) {
    size = Math.min(size, 100); // cap page size
    Pageable pageable = PageRequest.of(page, size,
        Sort.by(sort).descending());
    return orderService.findAll(pageable)
        .map(orderMapper::toDto);
}

// Repository
Page<Order> findByCustomerId(Long custId, Pageable pageable);
```

■ *Interview tip: Always cap page size server-side — allowing size=99999 can cause OOM / DB timeouts.*

Q38.**What is the difference between PUT and PATCH?**

- PUT: replaces the entire resource. Client must send the full representation. Missing fields are set to null/default.
- PATCH: partial update. Client sends only changed fields. Server merges changes.

```
// PUT – full replace
@PutMapping("/orders/{id}")
public OrderDto replace(@PathVariable Long id,
    @Valid @RequestBody OrderRequest req) {
    return orderService.replace(id, req); // all fields updated
}

// PATCH – partial update
@PatchMapping("/orders/{id}")
public OrderDto patch(@PathVariable Long id,
    @RequestBody Map<String, Object> fields) {
    return orderService.patch(id, fields);
}
```

■ *Interview tip: PATCH is idempotent in practice but not required to be by HTTP spec — a nuance worth mentioning.*

SECTION 6 — Microservices & Spring Cloud

Q39–Q43 | Circuit Breaker, Saga, Communication

Q39.**What is a Circuit Breaker and how do you implement it with Resilience4j?**

- CLOSED: normal operation — requests pass through.
- OPEN: failure threshold exceeded — fast fail with fallback. No calls to failing service.
- HALF_OPEN: probe state — allow limited requests to test recovery.

```
@CircuitBreaker(name="paymentService", fallbackMethod="payFallback")
@Retry(name="paymentService")
@TimeLimiter(name="paymentService")
public CompletableFuture<PaymentDto> pay(PaymentRequest req) {
    return CompletableFuture.supplyAsync(()->client.pay(req));
}

public CompletableFuture<PaymentDto> payFallback(
    PaymentRequest req, Exception ex) {
    return CompletableFuture.completedFuture(PaymentDto.queued());
}

# application.yml
resilience4j.circuitbreaker.instances.paymentService:
  slidingWindowSize: 10
  failureRateThreshold: 50
  waitDurationInOpenState: 15000
```

■ *Interview tip: Stack @Retry + @CircuitBreaker + @TimeLimiter together — that's production-grade resilience.*

Q40.**What is the difference between synchronous (REST/gRPC) and asynchronous (Kafka) communication?**

- Synchronous (REST/Feign): simple, immediate response, tight coupling, cascading failures possible.
- Asynchronous (Kafka/RabbitMQ): decoupled, resilient, eventual consistency, harder to debug.
- Use sync for: query operations, real-time user-facing responses.
- Use async for: order processing, notifications, ETL, event sourcing.

```
// Kafka producer – async event
@Service public class OrderService {
    @Autowired KafkaTemplate<String,OrderEvent> kafka;

    public Order placeOrder(OrderRequest req) {
        Order order = orderRepo.save(new Order(req));
        kafka.send("orders.created",
            new OrderEvent(order.getId(), CREATED));
        return order; // return immediately, don't wait
    }
}
```

■ *Interview tip: Outbox pattern solves dual-write (DB + Kafka) consistency — mention it for bonus points.*

Q41.**What is the Saga pattern for distributed transactions?**

- Choreography: services emit events, others react. Decoupled but hard to track flow.
- Orchestration: central Saga coordinates steps and runs compensating transactions on failure.

```
// Orchestration saga
@Component
public class OrderSaga {
    public void execute(Order order) {
        try {
            paymentService.reserve(order); // step 1
            inventoryService.reserve(order); // step 2
            orderService.confirm(order); // step 3
        } catch (PaymentException e) {
            // compensate – only payment failed
            paymentService.refund(order);
        } catch (InventoryException e) {
            // compensate both
            inventoryService.release(order);
            paymentService.refund(order);
        }
    }
}
```

■ *Interview tip: Axon Framework automates saga orchestration — mention it if asked about production tooling.*

Q42.**What is an API Gateway and what does it do?**

API Gateway is the single entry point for all client requests. It handles cross-cutting concerns so individual services don't have to.

- Routing: forwards requests to correct microservice.
- Authentication/Authorization: validates JWT before reaching services.
- Rate limiting: prevents abuse.
- Load balancing: distributes across service instances.
- Request/response transformation, logging, circuit breaking.

```
# Spring Cloud Gateway route config
spring.cloud.gateway.routes:
- id: order-service
  uri: lb://ORDER-SERVICE    # Eureka load balanced
  predicates:
  - Path=/api/orders/**
  filters:
  - StripPrefix=1
  - name: RequestRateLimiter
    args: {redis-rate-limiter.replenishRate: 10,
          redis-rate-limiter.burstCapacity: 20}
```

■ *Interview tip: Spring Cloud Gateway (reactive, Netty) is faster than Zuul (servlet-based) — mention the difference.*

Q43.**What is service discovery and how does Eureka work?**

Services register themselves with Eureka on startup with their host/port/health URL. Clients query Eureka to find service instances by name instead of hardcoded URLs. Eureka client caches the registry locally for resilience.

```
# application.yml (each service)
spring.application.name: order-service
eureka.client.serviceUrl.defaultZone:
  http://eureka:8761/eureka/

// Load-balanced HTTP call by service name
@Bean @LoadBalanced
public WebClient.Builder webClientBuilder() {
  return WebClient.builder();
}

webClient.get()
  .uri("http://payment-service/api/payments/" + id)
  .retrieve().bodyToMono(PaymentDto.class);
```

■ *Interview tip: In Kubernetes, use K8s built-in service discovery instead of Eureka — it's the cloud-native approach.*

SECTION 7 — Redis & Caching

Q44–Q46 | Spring Cache, Stampede, Distributed Lock

Q44.**How do you implement Redis caching in Spring Boot?**

```
# application.yml
spring.cache.type: redis
spring.data.redis.host: localhost
spring.data.redis.port: 6379

@SpringBootApplication @EnableCaching
public class App {}

@Service public class ProductService {
    @Cacheable(value="products", key="#id",
        unless="#result == null")
    public Product findById(Long id) { return repo.findById(id); }

    @CachePut(value="products", key="#product.id")
    public Product update(Product p) { return repo.save(p); }

    @CacheEvict(value="products", key="#id")
    public void delete(Long id) { repo.deleteById(id); }
}

// TTL config
@Bean RedisCacheConfiguration cacheConfig() {
    return RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(Duration.ofMinutes(30));
}
```

■ *Interview tip: unless="#result == null" prevents caching null results — avoids cache-null pollution.*

Q45.**What is cache stampede and how do you prevent it?**

Cache stampede (dog-pile): when a popular cache key expires, many concurrent requests all miss cache and hammer the DB simultaneously. Solutions:

- Mutex/lock: first miss acquires a Redis lock to rebuild; others serve stale or wait.
- Background refresh: scheduled job proactively refreshes hot keys before expiry.
- Probabilistic early expiry (XFetch algorithm): randomly refresh before TTL expires.

```
public Product getProduct(Long id) {
    Product cached = cache.get(id);
    if (cached != null) return cached;

    // Acquire distributed lock
    String lockKey = "lock:product:" + id;
    Boolean locked = redis.opsForValue()
        .setIfAbsent(lockKey, "1", 10, SECONDS);
    if (Boolean.TRUE.equals(locked)) {
        try {
            Product fresh = db.findById(id);
            cache.put(id, fresh);
            return fresh;
        } finally { redis.delete(lockKey); }
    }
    Thread.sleep(50); return getProduct(id); // retry
}
```

■ *Interview tip: Use Redisson's RLock for production distributed locking — handles network failures and auto-expiry.*

Q46.**What Redis data structures do you know and when to use them?**

- String: simple key-value, counters, session tokens, rate limiting.
- Hash: store object fields — user profiles, shopping carts. More memory-efficient than JSON string.
- List: message queues, activity feeds, recent items (LPUSH/RPOP).
- Set: unique tags, online users, deduplication. Supports union/intersection.
- Sorted Set (ZSet): leaderboards, priority queues, rate limiting with sliding window.

```
// String – rate limiting
redis.opsForValue().increment("rate:user:" + userId);
redis.expire("rate:user:" + userId, 1, MINUTES);

// Hash – cart
redis.opsForHash().put("cart:"+userId, productId, qty);

// ZSet – leaderboard
redis.opsForZSet().add("leaderboard", userId, score);
redis.opsForZSet().reverseRange("leaderboard", 0, 9); // top 10
```

■ *Interview tip: ZSet for rate limiting with sliding window is a common system design answer — practice it.*

SECTION 8 — System Design & Performance

Q47–Q50 | Idempotency, Optimization, Security

Q47.**How do you design an idempotent REST API?**

An idempotent operation returns the same result no matter how many times it is repeated. GET/PUT/DELETE are naturally idempotent. POST is not — make it idempotent with:

```
@PostMapping("/payments")
public ResponseEntity<PaymentDto> pay(
    @RequestHeader("Idempotency-Key") String key,
    @Valid @RequestBody PaymentRequest req) {

    return paymentRepo.findByIdempotencyKey(key)
        .map(p -> ResponseEntity.ok(toDto(p)))
        .orElseGet(() -> {
            Payment p = processPayment(req, key);
            return ResponseEntity.status(201).body(toDto(p));
        });
}
// DB: UNIQUE constraint on idempotency_key column
```

■ *Interview tip: Stripe uses idempotency keys for all payment APIs — it's the industry standard for financial systems.*

Q48.**How do you optimize a slow Spring Boot application?**

- Database: indexes, fix N+1, connection pool tuning (HikariCP maxPoolSize), read replicas.
- Caching: Redis for hot reads, HTTP response caching with ETag/Cache-Control.
- Async: @Async for non-blocking tasks, Kafka for fire-and-forget operations.
- JVM: right-size heap (-Xmx), use G1GC or ZGC, tune GC pause targets.
- Code: avoid eager loading, use DTO projections instead of full entities.
- Profiling: Actuator + Micrometer + Prometheus/Grafana to identify hotspots.

```
# Quick wins in application.yml
spring.jpa.open-in-view: false # critical – disables OSIV
spring.datasource.hikari.maximum-pool-size: 20
spring.datasource.hikari.minimum-idle: 5
spring.main.lazy-initialization: true # faster startup
```

■ *Interview tip: `spring.jpa.open-in-view=false` is the single most impactful JPA performance setting — always set it.*

Q49.**What are Spring Boot security best practices?**

- Never store plain passwords — always BCrypt with strength 12+.
- Use stateless JWT authentication — no server-side session state.
- Apply method-level security with @PreAuthorize for fine-grained access.
- Rate-limit authentication endpoints — prevents brute force.
- Set security headers: HSTS, CSP, X-Frame-Options, X-Content-Type-Options.
- Validate all inputs — prevent SQL injection, XSS, path traversal.

```
@PreAuthorize("hasRole('ADMIN') or #userId == authentication.name")
public UserDto getUser(Long userId) { ... }

// BCrypt
@Bean PasswordEncoder encoder() {
    return new BCryptPasswordEncoder(12);
}

// Security headers
http.headers(h -> h
    .frameOptions(fo -> fo.deny())
    .contentTypeOptions(Customizer.withDefaults())
    .httpStrictTransportSecurity(hsts ->
        hsts.maxAgeInSeconds(31536000)));
```

■ *Interview tip: Mention OWASP Top 10 awareness — it shows the security maturity expected at 3-year level.*

Q50.**How do you implement distributed rate limiting with Redis?**

Use Redis to track request counts per user/IP with an expiring key. Sliding window with Sorted Set is more accurate than fixed window with String counter.

```
// Fixed window – simple
@Service public class RateLimiter {
    public boolean allow(String userId, int maxReqs, int windowSecs) {
        String key = "rate:" + userId;
        Long count = redis.opsForValue().increment(key);
        if (count == 1)
            redis.expire(key, windowSecs, SECONDS);
        return count <= maxReqs;
    }
}

// Sliding window – accurate
public boolean allowSliding(String userId, int max, int windowSecs) {
    long now = System.currentTimeMillis();
    long windowStart = now - windowSecs * 1000L;
    String key = "slide:" + userId;
    redis.opsForZSet().removeRangeByScore(key, 0, windowStart);
    Long count = redis.opsForZSet().size(key);
    if (count < max) {
        redis.opsForZSet().add(key, String.valueOf(now), now);
        redis.expire(key, windowSecs+1, SECONDS);
        return true;
    }
    return false;
}
```

■ *Interview tip: Redis Lua scripts make rate limiting atomic — prevents race conditions between check and increment.*